AI-Assisted Backfill Onboarding and Data Quality Intelligence

Petabyte-Scale Financial Data Platform
Abhijit Sriram | Amazon Solutions Architect Interview Document
Note: Existing-platform figures are illustrative, not disclosed production data. Pilot targets are projected, not measured.

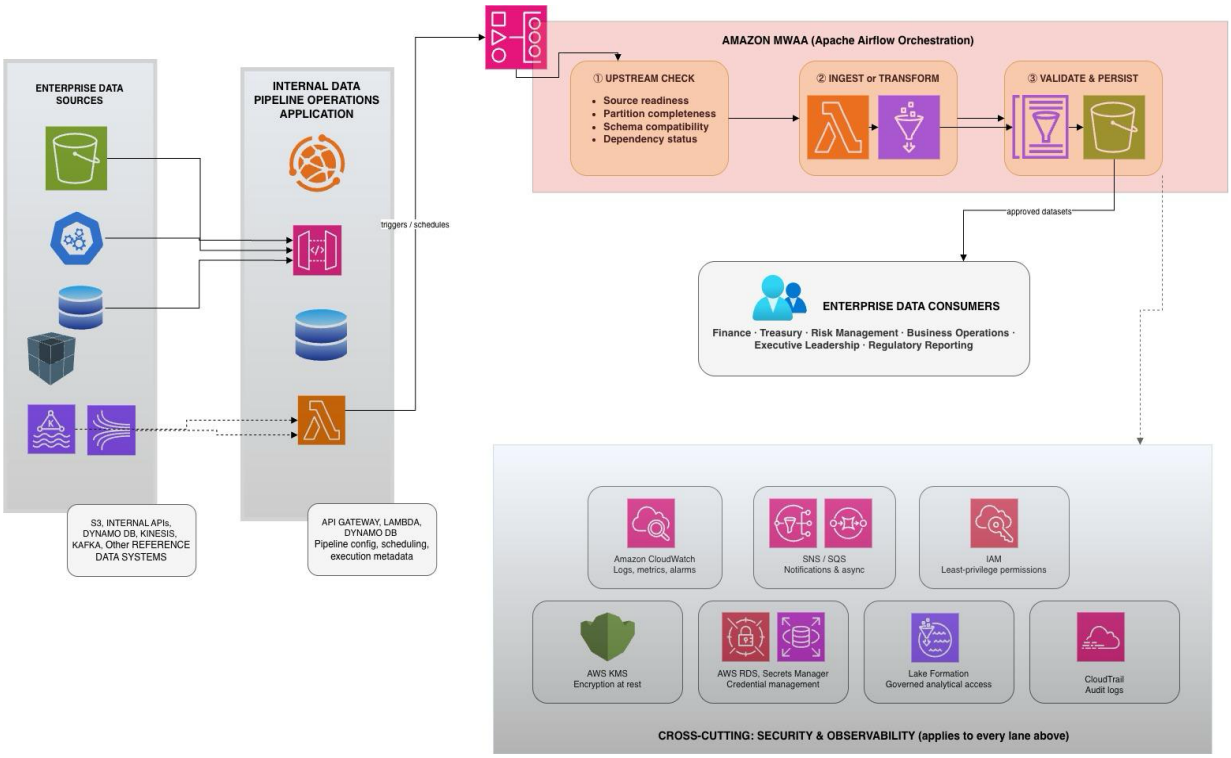
The Current Platform

Picture a large financial institution, spanning consumer banking, commercial banking, investment services, treasury, and corporate finance that consolidates data from across the business into one governed platform. At an illustrative scale, that means:

- Roughly 10 TB arriving per day, about 3.65 PB a year, with a historical footprint upward of 10 PB
- Around 300 workflows running on Amazon MWAA
- Data pulled from S3, internal REST APIs, DynamoDB, Kafka, and reference-data systems

Every workflow follows the same three-stage pattern. A pre-check confirms the right data exists before anything runs. Ingestion or transformation does the real processing work lightweight tasks on Lambda, heavier distributed work on Glue Spark or EMR. A final validation step reconciles the output before it's published into curated Iceberg tables and the Glue Data Catalog.

Security and governance encryption, access control, audit logging are already mature here. None of that is what this proposal changes; it's the reliable foundation everything else sits on top of.



The Business Problem

The platform is good at confirming that a pipeline followed its own rules. What it isn't built to catch is something subtler, and it shows up specifically when onboarding a new dataset. Onboarding almost always means backfilling historical data, and that history was never generated by one consistent system; it reflects whatever source system existed at the time, meaning it spans multiple generations of format, each needing its own mapping logic. There's no fixed shape to this problem either:

- The backfill window and the number of source-system eras involved are different for every dataset
- You can't pre-build one mapping pipeline and reuse it everywhere
- Hand-writing a mapping for every dataset, across every era it touches, doesn't scale once you're talking about hundreds of datasets

The deeper issue is that a backfill job can run cleanly, pass every schema check and reconciliation rule, and still be wrong. A historical period can look internally consistent while silently misrepresenting what happened. Deterministic checks can't catch a value that's plausible but incorrect.

The natural fix for the scale problem is to automate the mapping work with an LLM instead of writing it by hand but that introduces a new risk of its own: at this scale, no one manually reviews every individual mapping decision anymore, which is exactly where a quality problem could hide. Finance, Treasury, Risk, and Regulatory Reporting all consume this backfilled history as if it's already trustworthy, so a defect that slips through propagates into every report built on top of it. The opportunity, in one line: verify what the automation generates, not just whether the backfill job is technically completed.

PROPOSED SOLUTION PART ONE: AUTOMATED ONBOARDING AND MAPPING

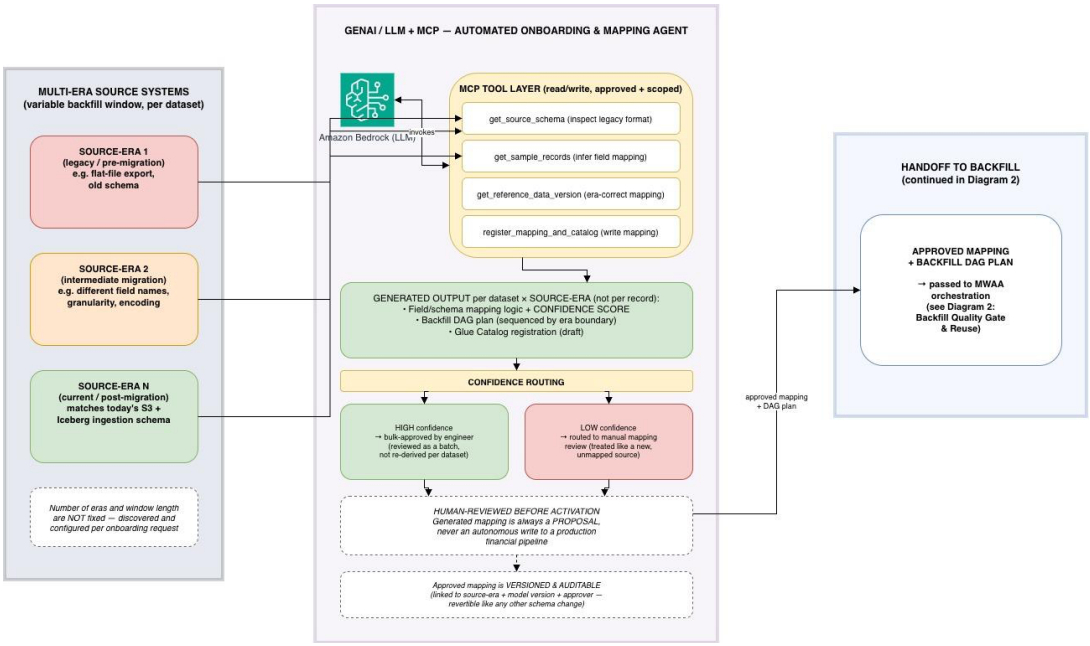
The first half of the solution tackles the scale problem without giving up verification. A GenAI agent built on Amazon Bedrock inspects each legacy source-era through a small set of scoped tools, using the Model Context Protocol:

- One tool reads the old schema
- One pulls sample records to infer how fields map
- One checks which version of reference data - cost-center mappings, currency tables were in effect during that specific historical period
- One registers the final mapping once it's approved

For each dataset and each source-era, the agent produces a proposed mapping, a confidence score, and a backfill plan sequenced by era boundary. That confidence score determines what happens next:

- High-confidence mappings are bulk-approved by an engineer as a batch, rather than reviewed one dataset at a time
- Low-confidence mappings are routed to full manual review, treated like a brand-new source

Either, nothing activates without a human sign-off. The mapping is always a proposal, never an autonomous write into a production financial pipeline. Once approved, it's versioned and auditable tied to the source-era, the model version, and the person who approved it — so it can always be traced or reverted later.



PROPOSED SOLUTION PART TWO: BACKFILL QUALITY GATE AND REUSE

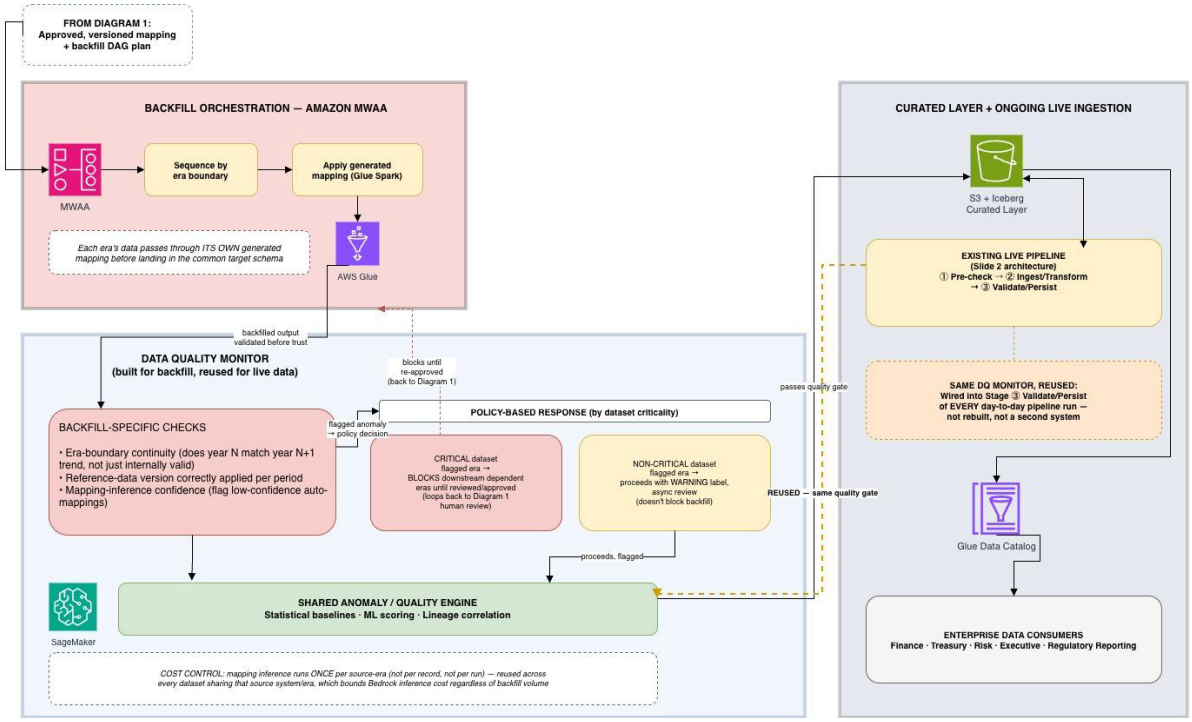
Once a mapping is approved, Amazon MWAA sequences the actual backfill by era boundary, applying each era's own mapping through Glue Spark before the data lands in the common target schema. The output then passes through quality checks that go beyond what schema validation can catch:

- Does this historical period trend consistently with the periods around it
- Was the correct historical reference data applied
- Does the mapping's confidence hold up against the real output

If something is flagged, the response depends on how critical the dataset is. A critical financial dataset blocks downstream processing until it's reviewed and re-approved. A non-critical one proceeds with a visible warning and gets reviewed asynchronously, so the whole backfill doesn't grind to a halt over a minor flag.

Underneath both paths sits one shared quality engine. Simple statistical baselines handle most datasets cheaply, while a smaller set of critical, multi-metric financial products use a trained model hosted on Amazon SageMaker, reserved for cases a simple baseline can't capture. Mapping inference itself runs once per source-era, not per dataset or per record; this is what keeps the cost of this approach bounded even at petabyte scale.

The most important architectural idea in this whole design is what happens next: that same quality engine isn't retired once the backfill is done. It's designed to be wired directly into the existing live pipeline's final validation stage, so the same capability protects both a one-time historical migration and everyday production monitoring going forward built once, not duplicated.



TECHNICAL DECISIONS AND TRADEOFFS

A handful of deliberate choices shape this design, and each one gave something up in exchange for what it gained:

- Human approval gate, not autonomy - slower, but a financial pipeline can't risk an ungated mistake reaching production.
- One shared engine, not two - avoids duplicate maintenance, but a bug now affects both the backfill and live-data paths at once.
- Policy by dataset criticality, not one uniform rule - avoids blocking too much or risking too much but requires someone to maintain that classification on an ongoing basis.
- Mapping generated once per source-era - keeps cost bounded, but a misidentified era boundary can affect every dataset that shares it.
- Synchronous checks for critical anomalies only, asynchronous for the rest - avoids making model latency a dependency for the whole platform, but a non-critical issue could reach consumers before its review finishes.

None of these decisions has one obviously correct answer. Each trades risk, cost, and operational exposure against the others, and the specific balance depends on the platform's own risk tolerance. Weighing that balance is the architectural judgment this project is about, not something that could simply be handed to an engineer as a fully specified task.

BUSINESS IMPACT AND PILOT TARGETS

Expected outcomes:

- Faster onboarding, since engineers review generated mappings instead of writing them by hand.
- Fewer undetected historical defects, since the new quality checks catch what deterministic validation alone misses.
- One capability serves two purposes, avoiding the cost of building and maintaining a duplicate system.

The specific numbers to validate in a pilot what percentage of mappings get approved without rework, what percentage of anomalies get caught against today's baseline of catching none, and how much faster review turnaround becomes are targets to measure, not results already achieved.

ROADMAP

The rollout is staged in four phases rather than deployed all at once:

- Phase 1: a small pilot on a handful of known-criticality datasets, to validate accuracy.
- Phase 2: an expansion phase that broadens coverage and tunes confidence thresholds based on what the pilot reveals.
- Phase 3: a scaling phase that extends the proven quality engine across the rest of the platform's live pipelines.
- Phase 4: a final phase that formalizes dataset criticality classification and approval ownership as a standing process, not a one-time setup.

CLOSING STATEMENT

This isn't a proposal to replace a platform that already works. It's a proposal to close the one gap that platform was never built to catch whether an automated, large-scale backfill produced correct data, not just data that happened to complete.